Swarm Learning and Knowledge Distillation Empowered Self-Driving Detection Against Threat Behavior for Intelligent IoT

Yibing Liu, *Member, IEEE*, Xiongtao Zhang, Lijun Huo, Jun Wu, *Senior Member, IEEE*, and Mohsen Guizani, *Fellow, IEEE*

***Abstract*—The combination of mobile communication and the Internet of Things (IoT) has made physical devices more intelligent, bringing great convenience to our lives. However, the deep integration of personal information and the Internet increases the risk of data leakage and is easily exploited maliciously. In addition, due to limited system resources, smart devices with lightweight design are required. Therefore, it is necessary to realize low-energy and effective abnormal behavior detection of IoT devices, but the existing detection methods have disadvantages such as leakage of user privacy, low accuracy, and difficulty in dynamically improving the effect. To address these issues, this paper proposes a dynamic interactive minor anomaly detection scheme called ADONIS based on Swarm Learning (SL). The scheme combines the concept of swarm defense and utilizes SL to achieve local data fusion, which improves the detection effect and protects user privacy. Moreover, the decentralized structure of SL can cope with the impact of single node damage to enhance the robustness of IoT services. Furthermore, we propose training and detection decoupling framework to achieve high accuracy, low energy consumption, and low latency. It improves the performance by fitting the training model with full data, and simplifies the complexity of the detection model using knowledge distillation. We also design a self-enhancing dynamic strategy based on the decoupling framework to maintain powerful detection capability through human-computer interaction (HCI) and continuous learning. The framework relies on traffic data to keep the model sensitive to new behavior through iterative training without disturbing the user. Finally, simulation experiments show that our proposed scheme can achieve 82.2% accuracy, reduce the average detection time to 8.22 ms, and simplify the model complexity by 15.9%. Compared with existing methods, ADONIS can provide lighter, safer and more accurate anomaly detection.**

Manuscript received 6 March 2023; revised 18 August 2023; accepted 30 October 2023. Date of publication 6 November 2023; date of current version 7 May 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 61972255 and in part by the Youth Fund of the National Natural Science Foundation of China under Grants 42201501 and 62303486. Recommended for acceptance by Z. Sun. *(Corresponding author: Jun Wu.)*

Yibing Liu is with the State Key Laboratory of Mathematical Engineering and Advanced Computing, Information Engineering University, Zhengzhou 450000, China (e-mail: 18121095425@163.com).

Xiongtao Zhang is with the College of Systems Engineering, National University of Defense Technology, Changsha 410072, China (e-mail: zhangxiongtao14@nudt.edu.cn).

Lijun Huo is with the Science and Technology on Information Systems Engineering Laboratory, National University of Defense Technology, Changsha 410072, China (e-mail: huolijun@nudt.edu.cn).

Jun Wu is with the Graduate School of Information, Production and Systems, Waseda University, Fukuoka 169-8050, Japan (e-mail: jun.wu@ieee.org).

Mohsen Guizani is with the Mohamed bin Zayed University of Artificial Intelligence (MBZUAI), Abu Dhabi 999041, UAE (e-mail: mguizani@ieee.org).

Digital Object Identifier 10.1109/TMC.2023.3330514

***Index Terms*—Human-computer interaction, IoT, network abnormal detection, swarm defense, training-detection decoupling.**

I. Introduction

With the continuous development of mobile communications, we are embracing the era of the intelligent IoT. By 2025, the number of IoT devices is expected to reach 38.6 billion [1]. They are increasingly used in smart homes, health care, intelligent vehicles, smart city, industry control and other areas. The IoT, connecting people through a large number of mobile communication service and applications, are under rapid development. At the same time, the IoT devices based on 5G have so many security vulnerabilities, such as weak passwords, unpatched software, lack of anti-virus means and 24-hour online, which make them so attractive for attackers [2]. Consequently, the number and means of network attacks are also increasing rapidly. IoT devices-relevant attacks (e.g., denial of service, false data injection attacks in smart grids) [3] and attacking critical infrastructure using a botnet of home IoT devices may cause a disruption of infrastructure service and result in significant financial losses. Therefore, it is imminent to build an effective protection system for IoT devices. Network behavior anomaly detection based on traffic analysis is increasingly becoming a new means of network security protection. Usually, the traffic analysis uses Machine Learning (ML) methods to analyze sequence data, mainly focusing on supervising and semi-supervising models, and has achieved some success. Moreover, it also faces some specific problems. For example, the network traffic is too complex to analyze, and attack traffic samples are so small that it is difficult to extract useful features [4].

In the IoT environment, edge nodes are widely distributed, and a single node's behavior mode is simple, significantly reducing the difficulty of traffic feature extraction and increasing the difficulty of obtaining the number of samples. Meanwhile, the IoT equipment has a great demand for energy consumption optimization, making its micro-architecture unsuitable for extensive firewalls and traditional ML methods. So, efficient detection is challenging. At the same time, how to combine abnormal behavior detection with daily network security protection is also a field that few people have stepped into at present. Traditional abnormal behavior detection is usually carried out independently by a single device because the collected data packets often

1536-1233 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: King Mongkut's University of Technology Thonburi. Downloaded on October 13,2026 at 10:16:10 UTC from IEEE Xplore. Restrictions apply.

contain sensitive information and cannot be uploaded to the central server for centralized processing, resulting in inefficient judgments. As discussed in the work of Kolias et al. [5], Mirai and malware bots can exploit IoT device vulnerabilities to control their AI functions and further access other non-IoT devices connected to them. This underscores the fact that unprotected IoT devices may become a genuine threat to all other connected devices.

IoT devices have different structures and need frequent training to maintain optimal detection performance. Therefore, configuring an ML-based anomaly detection method becomes very difficult [6], [7], and might result in some attacks against open interfaces of users or developers. Over the years, many researchers have focused on solving open vulnerabilities of IoT devices [8], [9]. Some scholars, based on ML, have proposed Federated Learning (FL)-based solutions [10], [11], [12]. For this solution, dispersed ML model training keeps the data intact on edge devices. The learned ML parameters are transferred to a central server for aggregation. However, subsequent studies have also proved that FL faces some performance challenges [13], because the training results of each device are aggregated in the central server, which poses security threats to the whole structure. At the same time, IoT devices in FL usually have different structures. The model cannot accurately define all the device categories, leading to a severe decline in parameters. Recently, Wang et al. [14] has solved the device recognition problem, but it also brings new problems. An effective anomaly detection model needs to capture and distinguish all benign and malignant behavior patterns in time to define new device behaviors and add them to the model to maintain its function. Thus, we propose the concept of self-driving swarm defense based on the novel SL [15] framework, which is very suitable for solving some apparent problems. SL can collaboratively train small sample models as a new distributed learning paradigm. Because there is no single central learning node, it will not cause excessive load on the limited bandwidth of IoT devices. It can effectively reduce network attack threats caused by a single central training node. More importantly, the dynamic learning framework proposed by SL is very suitable, considering that IoT devices' abnormal behavior detection needs the ability to capture, distinguish and define new behavior patterns in real time. Traditional statistical ML methods need large amounts of data. Even combined with FL parameter learning functions, they are still unable to do anything once the attacker changes its behavior strategy and pattern, which means that the model can do nothing about the new characteristics. The self-driving swarm defense method combined with SL is different. It can identify new abnormal behavior by interacting with users and adding them to auxiliary samples for dynamic training to adapt to changes in aggressive behavior. At the same time, the attack features of any device can be shared globally so that other nodes can perceive the attack characteristics in advance and intercept them before they are attacked.

Although the above method settles down several critical problems, it is undeniable that these behaviors will significantly increase the time and power consumption of the detection model. Fortunately, IoT traffic usually differs from other internet-connected devices, such as laptops, mobile phones, and other terminals with complex communication [16]. IoT communications are usually limited endpoint communications rather than many web services. It has more repetitive network traffic. It transmits packets using the same protocol at fixed time intervals, and Doshi et al. [17] has summarized suitable high-accuracy ML methods. We can use the knowledge distillation method to extract the knowledge and merge redundant parameters to reduce power consumption, storage cost and communication delay.

Based on the above analysis, this paper proposes a lightweight, self-driving IoT traffic anomaly detection scheme which is based on SL, knowledge distillation and HCI and can be deployed in the home edge gateway. Its concept is shown in Fig. 1. The main contributions of this paper are as follows:

- This paper proposes the ADONIS scheme to achieve device abnormal behavior detection and fast response without interfering with IoT services. Considering the distributed nature of multi-connectivity in IoT, we design SL-based malicious traffic sensing architecture, which realizes data fusion of multi-device by sharing only the parameters without transmitting the raw data, both improving the accuracy of the model and avoiding the leakage of user's privacy. In addition, the decentralized structure of SL is not constrained by the damage of a single node, strengthening the stability of the detection service.
- We develop training and detection decoupled deployment mode to obtain detection models through full data training and knowledge distillation simplification. This detection model has lower latency and smaller size, which can meet the demand of lightweight and fast response of IoT devices.
- We innovatively introduce HCI mechanism to provide user-senseless self-driving data update through artificial + intelligence. ADONIS utilizes the new data set to dynamically train to improve the detection performance, and maintain sensitivity to unknown abnormal behaviors, thus enhancing the information security of IoT.
- Home IoT-based experiments show that ADONIS reaches a detection accuracy of 82.2%, reduces the average detection latency to 8.22 ms, and simplifies the model complexity to 15.9%, providing significant performance and efficiency advantages over other state-of-the-art feasible solutions. Meanwhile, our scheme can utilize continuously updated datasets to maintain high detection capability, proving the validity of our strategy.

Outlines. The rest of the paper is organized as follows. In Sections II and III, we describe the research progress of abnormal behavior detection and the related basic techniques involved in this paper. We present the ADONIS scheme in Section IV. Finally, we evaluate the effectiveness and performance of ADONIS in Section V, and conclude in Section VI.

II. Related Work

This section introduces the relevant research on abnormal behavior detection in IoT, mainly including two traffic analysis methods: centralized machine learning and distributed federated learning.

Authorized licensed use limited to: King Mongkut's University of Technology Thonburi. Downloaded on October 13,2026 at 10:16:10 UTC from IEEE Xplore. Restrictions apply.

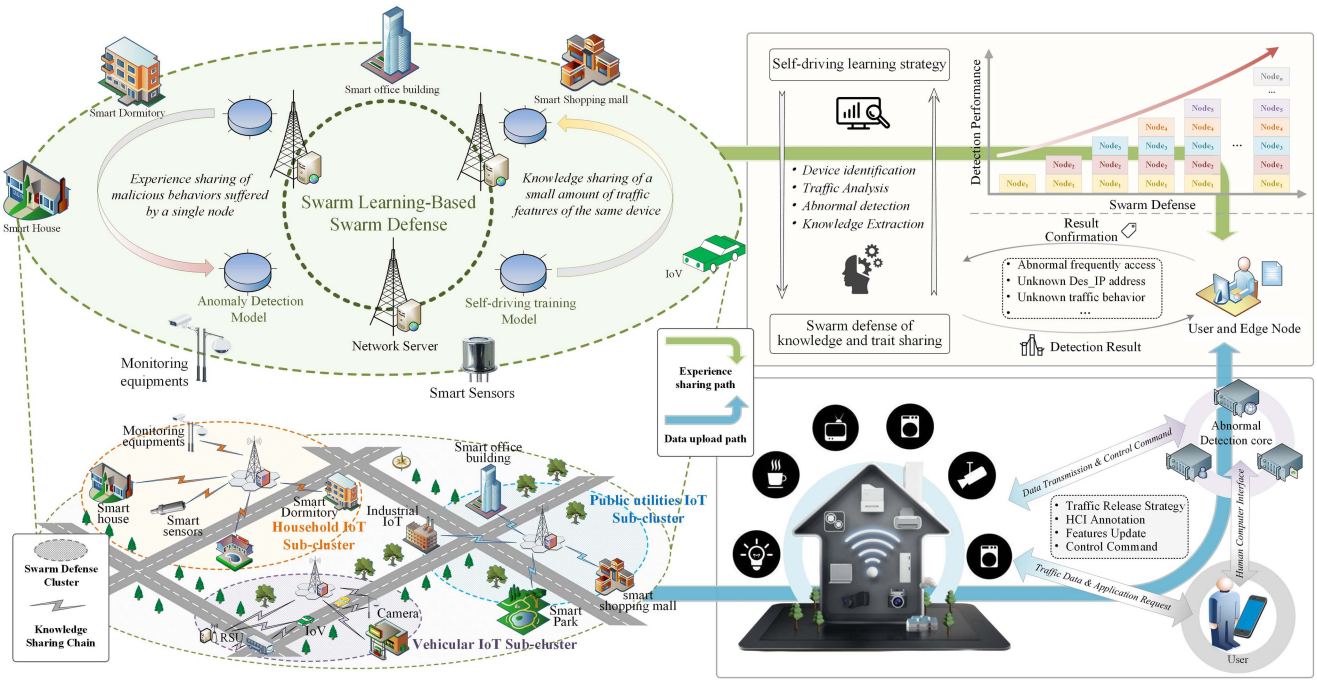


Fig. 1. Applicable scenarios of swarm defense-based self-driving ADONIS scheme. The left side mainly depicts the IoT anomaly detection scenario that this paper focuses on and its perfect compatibility with the SL framework. Each sub-cluster in IoT can be regarded as a swarm node. When these nodes are effectively connected, the entire IoT is abstracted into a swarm network, which has the ability of cluster defense. The right side briefly summarizes the security mechanism of ADONIS scheme design and their mutual interaction with specific IoT devices.

There are many machine learning-based anomaly detection algorithms [18], such as deep auto-encoder that have been proven to successfully detect abnormal network traffic from IoT devices [19], and clustering is another effective technique to find deviation from normal behavior [20]. However, the algorithm complexity of these two methods is high, which will increase the detection cost. In addition, after testing a variety of ML methods, Doshi et al. [17] conclude that random forests, k-nearest neighbors, and neural networks are very effective for data classification, and gives feasible ideas for abnormal behavior modeling. However, these methods depend on specific scenarios. Its effect is not stable. To improve detection performance, Sivanathan et al. [21] develop a ML framework for device classification and traffic analysis based on features such as port occupancy, protocol patterns, and packet structure, achieving high accuracy rates. Rezvy et al. [22] propose a hybrid anomaly detection system based on deep learning, focusing on the balance between efficiency and accuracy. It utilizes auto-encoder learning through a dense neural network framework to simultaneously achieve low detection cost and high detection performance. And, Xiao et al. [23] improve the detection accuracy by sharing device behaviors securely to cloud server, and develop a dynamic game-based Q-Learning anomaly sensing scheme that utilizes the Dyna architecture and radio channel model to enhance its performance. In addition, Weinger et al. [24] propose to use data augmentation to enhance the performance of anomaly detection, and found that stratified and uniform random sampling performed particularly well. Data augmentation makes the model more generalizable and able to adapt to more IoT scenarios. However, the above models are all centralized data analysis methods, which need to collect original user information, which will lead to the risk of data leakage.

To avoid centralized abnormal detection, Nguyen et al. [25] propose a self-study system called DIoT, which is a method based on FL and used for detecting IoT devices infected with Mirai malware. Similarly, Zhao et al. [26] also use FL to detect suspicious network data streams with the ability to identify adversarial malicious clients in specific application scenarios and remove them from the network. These two approaches are based on multiple IoT devices to achieve joint detection, which is a novel idea. In addition, Liu et al. [27] apply the Convolutional Neural Network (CNN) model and short and long-memory development to improve the accuracy of abnormal behavior detection while using gradient compression to reduce communication costs and improve communication quality in the FL process. Yi et al. [28] propose to enhance the LSTM and CNN models using attention mechanism to improve the detection accuracy. Phan et al. [29] have used software-defined networks to monitor network traffic at stream-level granularity. Their work shows that flow-based features can effectively detect security threats without the high computational overhead of deep packet inspection. Ghimire et al. [30] adopt FL to detect suspicious network traffic while protecting data privacy by identifying hostile malicious clients in specific application scenarios and removing them from the network. Mothukuri et al. [10] utilize federated training wheels on the GRUs model and shared the learning weights with FL's central server to keep the data intact and optimize the accuracy of the global model. The above method adopts a distributed ML model, which doesn't need to transmit the original data, and only needs to share

Authorized licensed use limited to: King Mongkut's University of Technology Thonburi. Downloaded on October 13,2026 at 10:16:10 UTC from IEEE Xplore. Restrictions apply.

the intermediate parameters of the model to realize abnormal detection, which greatly improves information security. It has significant advantages in privacy protection and has excellent accuracy for attack detection. However, the FL-based model may put the whole system at attack risk due to the damage of the central node. There has been a lot of work on the traditional Internet traffic analysis problem [31], [32]. However, unlike the large-scale communication of the Internet, the traffic generated by a single gateway of the IoT is small, and only the total traffic generated by hundreds or thousands of devices can be counted [33]. This observation is also confirmed in the work [34].

This paper uses the combination of SL and deep learning to complete the anomaly detection of the IoT, which has many advantages. First, data analysis based on deep learning can fully realize information fusion and bring higher detection performance to the model. Second, we adopt a decentralized distributed learning framework based on the SL, which can not only avoid the security risks caused by centralized data analysis, but also solve the instability problem caused by the central node built by FL. Third, we also adopt knowledge distillation technology to simplify the model structure and reduce parameters, which greatly improves the cost of anomaly detection and makes the model suitable for a wide range of high/low power consumption scenarios.

III. Preliminary

A. Swarm Learning

SL [15] is a new ML method that integrates edge computing and blockchain technology. SL provides a learning framework that uses a distribution-before-integration training pattern to aggregate data scattered across multiple nodes to improve the final performance of the model. Given a certain basic model, SL first makes edge computing node train independently based on local data. After that, the parameters are shared with a temporary central node through blockchain technology. When the parameters of all nodes are collected, the central node aggregates the parameters in some way, such as weighted average, to form the final model. Then the central node will distribute the aggregated model to each edge node through the blockchain network to complete anomaly detection task.

Unlike FL [35], the parameter upload of SL relies on blockchain technology. It uses SL to maintain data connectivity and provide distributed authentication services among nodes. Because there is no central node, it can still operate normally and keep stable when a node fails. The entire behavior detection cluster of the IoT is composed of a large number of edge nodes. Each node's local data involves user privacy and is unsuitable for sharing. Moreover, the dynamic learning method proposed by the SL framework is also very suitable for after-deployment learning of traffic that is constantly updated.

B. Knowledge Distillation

Knowledge distillation is one of the widely accepted model compression methods, which aims to endow a new network with the knowledge hidden in the trained large-scale model so that the new network gets the same ability. The new network is termed as 'student', and the original network is termed as 'teacher'. Usually, a student network is much smaller than the teacher, while the student can carry close knowledge to the teacher. The process of training a small-size student to keep approaching the input and output of the large-size teacher model is called knowledge distillation. Because of its high efficiency and scalability, knowledge distillation has been successively used since it was proposed by Hinton [36], in image recognition [37], text processing [38] and other fields, and achieved relatively good results.

This paper combines knowledge distillation to accomplish the decoupling of training and detection. Knowledge distillation allows small-scale model learn knowledge based on large-scale model to achieve the accuracy that small model cannot achieve alone. During the training process, we apply the large-scale model based on the traffic data set for parameter learning. However, although the structure of the large-scale has been optimized, it is still too complex for lightweight IoT devices. Therefore, the knowledge distillation is applied in the detection process to prune the model and further simplify the network structure to reduce its complexity. We decouple the training and detection process through large-scale learning and knowledge distillation pruning. The advantage of this approach is that it enables the model to learn from all traffic data, improving the accuracy of anomaly detection while reducing the complexity of the algorithm and making the ADONIS scheme applicable to lightweight devices. It is worth pointing out that the training process relies on numerous edge nodes integrated by the SL framework, while the detection process is done on a single node.

IV. ADONIS Scheme

This section mainly describes the SL-based lightweight, self-driving IoT traffic anomaly detection scheme, and the overall process is shown in Fig. 2. The scheme mainly contains three modules: feature capture and device recognition IV-A, abnormal behavior detection core IV-B, and self-driving dynamic learning strategy based on HCI??. Where IV-A and IV-B constitute the detection core in Fig. 2 and ?? undertook the swarm defense and self-driving learning strategy. Specifically, we first collect network traffic packets of IoT devices from gateways deployed at the network's edge and extract characteristic data according to the type of devices. We need to input their traffic packets into a classifier for new devices to identify their type. Then, all the device's network traffic will go through the abnormal behavior detection process. The packet segment is released from the gateway if the result is normal. Otherwise, the authentication process is triggered. Based on the authentication results, we decide whether the packet segment can finally be released from the gateway. At the same time, we annotate the results as feature data in the sample database. The sample database will regularly train the training model deployed on the edge nodes. This can achieve a complete version of the self-driving dynamic learning so that the solution can be continuously adjusted and improved according to the actual network behavior after deployment. In addition, we use SL for knowledge sharing and take advantage

Authorized licensed use limited to: King Mongkut's University of Technology Thonburi. Downloaded on October 13,2026 at 10:16:10 UTC from IEEE Xplore. Restrictions apply.

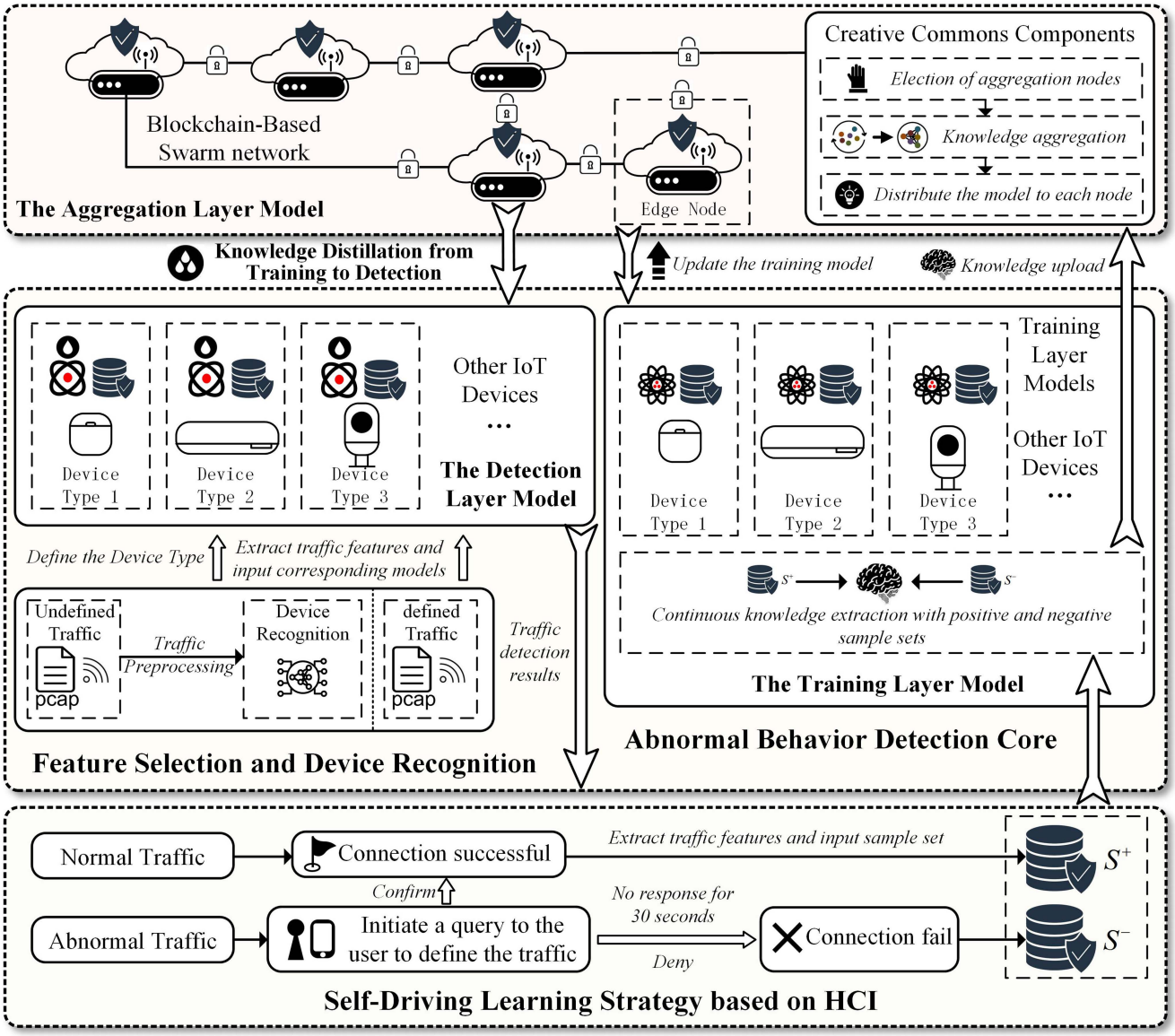


Fig. 2. ADONIS scheme architecture containing three main modules. We take three IoT devices as examples to introduce the execution process of the ADONIS scheme. The right part mainly describes the operation mode of the training layer, which trains the model based on local data and shares the fitted parameters to the blockchain-based swarm network. The left part mainly describes the operation mode of the detection layer, which obtains the integrated model generated by the swarm network by aggregating all local model parameters. The bottom is the HCI-based self-driven dynamic learning strategy, which can dynamically improve the model effect through the update of the data set.

of distributed and data fusion to improve the detection effect and safety of the model. We also innovatively use knowledge distillation technology to condense the scale of the model, reduce the cost, and improve efficiency while ensuring the detection effect.

A. Feature Selection and Device Recognition

1) Device Type Recognition: Device recognition can rapidly and accurately define IoT devices. The first step in IoT abnormal behavior detection is to accurately identify the device type to achieve preprocessing of raw traffic data. However, smart devices are usually produced from multiple sources and don't have a unified identification standard. Therefore, the IoT cannot simply confirm an unknown device type through traditional identity information, such as MAC address. Fortunately, Sivanathan et al. [21] utilizes statistical attributes such as activity periods, port numbers, and cipher suites to summarize the characteristics of devices' network traffic. They have developed a multi-stage intelligent recognition method for IoT devices. Experiments show that this method can accurately identify new devices at a low cost and without other equipment or protocol. We add this method to our detection system to accomplish initial device recognition.

Specifically, abstracting the method of Sivanathan et al. [21] into a classifier model C, we introduce a device recognition Algorithm 1. For an unknown device set $d_i (i = 1, 2, 3, \ldots)$, we first collect its network behavior data and extract identity

Authorized licensed use limited to: King Mongkut's University of Technology Thonburi. Downloaded on October 13,2026 at 10:16:10 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Device Recognition Algorithm.

Require: A new device set $\mathbf{D} = \{d_1, d_2, d_3, \ldots\}$, classifier model C, device type set $\mathbf{T} = \{t_1, t_2, t_3, \ldots\}$
Ensure: Get the type of the new device set
set result set: $\mathbf{R} = None$
for each i **do**
  Collect network behavior packets p_i of d_i
  Extract device identification features s_i from p_i
  $flag$=$C(s_i)$, and $flag \in \mathbb{R}^+$
  set type of d_i to t_{flag}
  $\mathbf{R}.add((d_i -> t_{flag}))$
end for
$\mathbf{D}$=$\mathbf{R}$
return $\mathbf{D}$

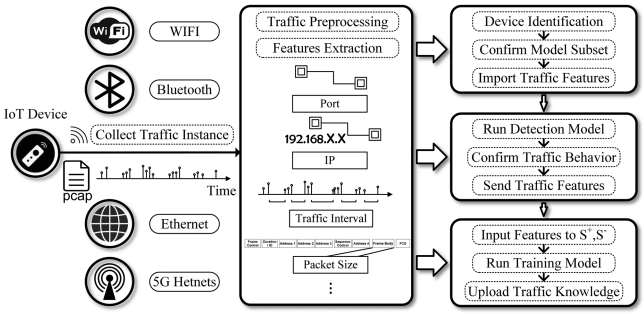


Fig. 3. Extraction and processing of IoT device traffic features.

features. And then we input the feature information into the classifier C to obtain the type of the device, and finally assign a specific type to the corresponding unknown device. Ultimately, we can acquire a fresh set $\mathbf{D}$ labeled with device type.

2) Packet Network Traffic Feature Selection: As shown in Fig. 3, to capture the traffic feature data of all IoT devices in real-time, we set each gateway as an edge node and deploy boxes in the network. These boxes are designed to grab network packets and divide them according to time windows, which are simple representations of a device's network behavior. Then we aggregate the statistics of multiple data packets in the gateway to extract the corresponding feature data to be used as input for device identification, abnormal behavior detection, and positive and negative sample sets. To map the network feature from real discrete space to a vector space that the neural networks can process, one-hot and normalized mappings are used before pre-processing.[1] In experiments, we explored the effectiveness of feature data in improving computational results after entering the model. We extracted some traffic features with significant behavioral differences between normal and abnormal traffic.

[1]Specifically, a one-hot form was transformed into class-specific vectors for the source port, target IP (hash), target port, protocol, service type, connection state, and DNS query type features. For quantified features such as source IP capacity, target IP capacity, source packets, target packets, sending information, receiving information, duration, time interval, and samples, a normalized mapping is used, i.e., $\hat{f}_i = \frac{f_i - \min_i f_i}{\max_i f_i - \min_i f_i}$, where f_i is the original feature, and $\hat{f}_i$ is the normalized feature.

3) Packet Size and Number of Requests per Unit Time: In general, there are significant differences between the attack and normal traffic in terms of packet size and the number of requests per unit of time because attackers usually want the victim open as many connection requests as possible to consume the victim's server resources or make the connection successful. Therefore, the attacker expects the packet size to be as small as possible to maximize the number of connection requests per second. According to the data collected in the past and the research of Doshi et al. [17], more than 90% of the attack packets are below 100 bytes. Most packets vary between 100 and 1200 bytes.

4) Inter-packet Interval: Normal IoT traffic is far from active, with most packets being sent regularly and a considerably long time between packets. This may be seen as a reflection of the IoT network activities or other automated network activities. In contrast, sniffing and control activities may sometimes cause frequent operations.

5) Protocol: In general, normal traffic and attack traffic use different communication protocols. UDP hosts most video streams. Control operation is usually realized via TCP protocol. So, UDP usage is generally more than three times TCP usage. Similarly, the usage of unusual protocols can accurately help detect anomalous behavior.

6) IP, Port Destination Address Cardinality and Specificity: IoT devices are featured in limited communication ports [39]. For example, an OPPLE Smart Ceiling Light communicates via only four ports to activate/deactivate, retrieve firmware updates, and record their status from the cloud. Another critical feature of an IoT device traffic is that the set of destination IP addresses rarely changes over time. In general, packets associated with attack traffic are connected to more ports on average, so the presence of many ports per unit of time or significant changes in the number of destination IP addresses can clearly indicate anomalous behavior.

7) Bandwidth: Past research proves that bandwidth usage can be used to characterize IoT device network traffic. For example, Apthorpe et al. [39] described the usage pattern of consumer-grade IoT devices by the send/receive rate. The instantaneous bandwidth of the device can be calculated by measuring the size of data packets transmitted per unit of time, and the subtle bandwidth distribution difference can help to distinguish whether the device behavior is normal.

B. Device Abnormal Behavior Detection Core

As shown in Fig. 2, the device abnormal behavior detection model can be divided into three parts: feature selection and device recognition, device abnormal behavior detection core, and self-driving learning strategy based on HCI. This section mainly describes the second modules, namely the core implementation. The detection and training layer are preset with sub-models corresponding to each device d_i. In the pre-deployment stage, the detection layer and the training layer have detection model DM_i and training model TM_i corresponding to different equipment types. The complete teacher model is stored in the training layer and can be called by SL to train the aggregation model dynamically. SL selects appropriate nodes for aggregation in

Authorized licensed use limited to: King Mongkut's University of Technology Thonburi. Downloaded on October 13,2026 at 10:16:10 UTC from IEEE Xplore. Restrictions apply.

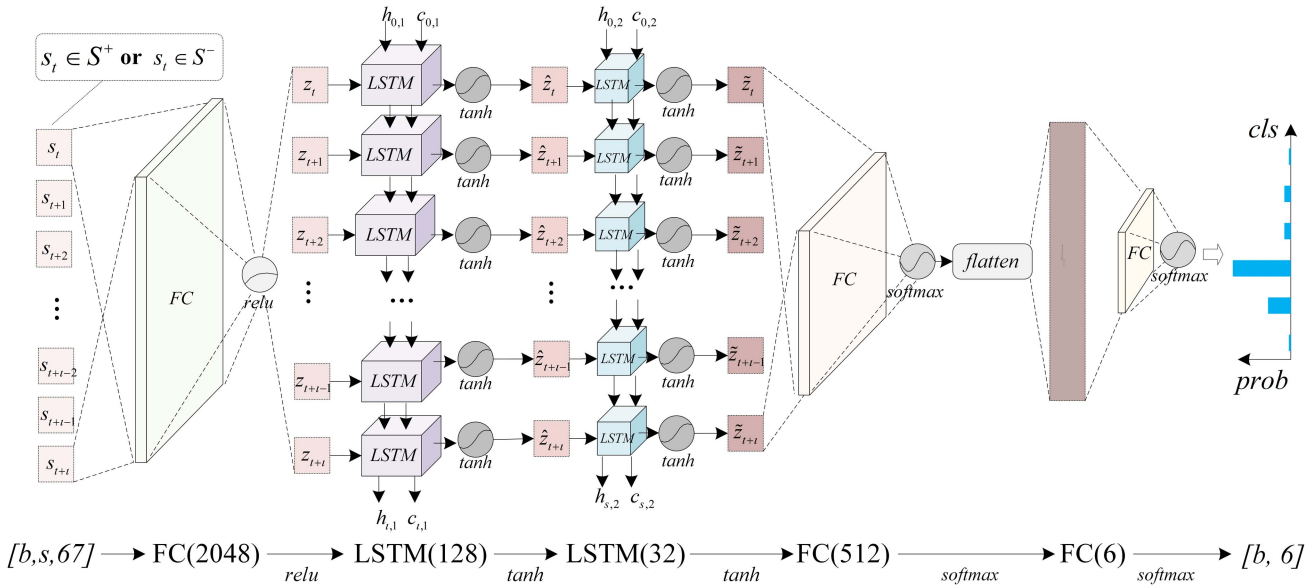


Fig. 4. Structure of training layer models.

each dynamic learning process to get the updated teacher model TM_{i+1}. After downloading the dynamically trained aggregate teacher model TM_{i+1} from the SL layer, the detection layer distills the model into a student network (used as a detection model DM_{i+1}) to perform efficient detection tasks. The specific structure and parameters of the model and the knowledge distillation process are as follows.

1) The Training Layer Model: After capturing the packet's segment feature, the abnormal behavior detection can be performed by using these extracted features. When the computational consumption of the edge nodes are at the bottom, the complete model is trained. To fully tap the temporal relationship between data features, positive samples s_t^+ and negative samples s_t^- are collected, and time sequence stitching is carried out. The sample characteristics of the same device d_i are spliced according to the time sequence relationship, and the following results are obtained:

$$\overline{s}_t = concat(s_t, s_{t+1}, \ldots, s_{t+\iota}), \tag{1}$$

where ι represents the sequence length, $concat(\bullet)$ is the concatenation operator, $s_t \in S^+ \bigcup S^-$. For example, for a sample with an original dimension of 67, its feature is s_t. If $\iota = 32$, then the dimension of $\overline{s}_t$ is $[32, 67]$. For the preprocessing before inputting into model, see Section V-B.

Then, these time series data and their implicit knowledge are learned through the training layer model TM_i. The training layer model uses stack Long Short-Term Memory (LSTM) [40], which is a recurrent neural network that can learn the long-term dependence of data. And it is mature and has been widely validated, applied and analyzed, the composition of the anomaly detection model is based on LSTM. The structure of LSTM is shown in Fig. 4. After inputting 2048 units fully connected network in the time sequence sample $\overline{s}_t$, it is activated by $relu$. The result is processed by 128 unit LSTM and 32 unit LSTM, and then processed by 512 unit fully connected layer. Then through the flattened splicing, there is a fully connected network to output the results and get the probabilities corresponding to different types. For recording purposes, the network structure in the map is $f(\theta_{T_i}; \overline{s}_t)$, and θ_{T_i} is the model parameter of TM_i.

2) The Aggregation Layer Model: This module is implemented through the SL framework based on blockchain technology. Blockchain is a data organization structure. The communication in the SL is mainly implemented through blockchain, which enables secure and effective parameter sharing. Due to the decentralized and distributed structure of the blockchain, the ADONIS scheme based on the SL can maintain the normal operation of other nodes even when a node is under attack, which greatly improves the security and stability of the system. In addition, the openness of the blockchain facilitates the dynamic addition and deletion of edge nodes and gives good scalability to the SL, enabling the ADONIS to provide effective detection and security protection in the face of abnormal or new devices.

Specifically, the module first splices the samples in the positive and negative sample sets by way of (1). The network model corresponding to each device type in the edge node can be updated locally. It is realized like this:

$$\theta_{T_i}^{(e+1)} = \theta_{T_i}^{(e)} - \eta \nabla L_{em}(\overline{s}_t, y_t; \theta_{T_i}^{(e)}), \tag{2}$$

where $\theta_{T_i}^{(e)}$ represents the model parameter of TM_i when local updates are conducted for e times, η represents learning rate. And $L_{em}(\overline{s}_t, y_t; \theta_{T_i}^{(e)})$ is the experience loss between the inference results and the ground truth after the original data ($s_t \in S^+$ or $s_t \in S^-$) of the time sequence sample $\overline{s}_t$ is judged by the parameter $\theta_T^{(e)}$.

Since the model parameters of a node may be harmful for performance, we reserve only the beneficial part of them as the rising parameter. Given the model parameters $\theta_{T_i}^{(e)}$ obtained by node i after e times of training based on local data, we treat the corresponding parameters as the rising parameter P_i^t if i has better anomaly detection. We execute the above operation for each node by loop iteration and acquire the rising parameter set

Authorized licensed use limited to: King Mongkut's University of Technology Thonburi. Downloaded on October 13,2026 at 10:16:10 UTC from IEEE Xplore. Restrictions apply.

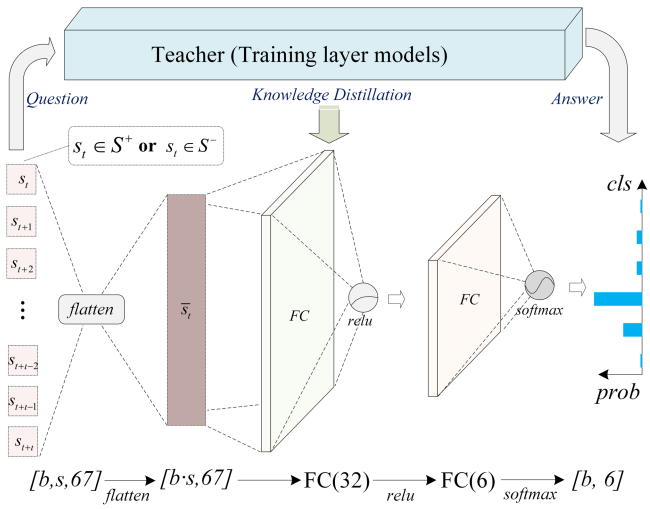


Fig. 5. Structure of detective layer model with knowledge distillation.

$P = \{P_1^t, \ldots, P_i^t, \ldots, P_n^t\}$. After that, all nodes jointly upload P to the SL aggregation node to integrate the rising parameters. In this paper, we use the same aggregation method as the work of Warnat-Herresthal et al. [15], i.e.

$$TM_{t+1} = \sum_i \frac{n_i}{n} P_i^t. \tag{3}$$

where n_i refers to the sample size adopted by the node corresponding to the rising parameter P_i^t during training. This is consistent with the strategy taken in the work of Yang et al. [35] and Warnat-Herresthal et al. [15]. The after-convergence training model TM_{t+1} is shared with each edge node through the SL. The edge nodes then replace local model parameters θ_{T_i} to $\theta_{T_i}^{(0)} = TM_{t+1}$ after they get TM_{t+1}. Subsequently, the large model starts a new round of iterative training in the training layer.

3) The Detection Layer Model: To shorten the abnormal behavior detection period at the edge gateway and accelerate the detection speed while ensuring high-quality performance, we use the method of knowledge distillation to transfer the knowledge of the trained model TM_i to a smaller network DM_i and then solve the contradiction between model size and recognition accuracy.

To make inference as fast as possible, the network DM_i uses a lightweight multi-layer full-connection module; its model structure is shown in Fig. 5. In this structure, the time series data is processed by flatten, then by 32 unit fully connected network and 6 units fully connected network respectively, and then get the model results. Due to the small scale of the student network, its ability to train the model directly from data is low. However, the knowledge learned by the TM_i is distilled into the DM_i, i.e., the "student" inherits the training results from the "teacher". The "teacher" network is accurate for the "student" network. DM_i should behave as much as possible like TM_i to ensure that it inherits sufficient knowledge and presents excellent detection ability. In addition, we can normalize the logits of the model's final layer by softmax to obtain the output of each classification, so as to evaluate the confidence of TM_i. And DM_i can effectively reduce the inference time and enhance the QoS of IoT. It's worth noting that the proposed framework is a model-free approach in which the network structure and style of TM_i and DM_i can be changed as needed.

4) Knowledge Distillation From Training to Detection: After putting the knowledge of TM_i to DM_i network, we use each edge node's $S = S^+ \bigcup S^-$ as the proxy data (data label is not necessary). Then, we take a batch of samples, make inferences with TM_i and DM_i, and get the logit result.

$$\begin{cases} l_{T_i} = f(\theta_{T_i}; s) \\ l_{D_i} = f(\theta_{D_i}; s) \end{cases}, s \in S, \tag{4}$$

where $f(\theta_{T_i}; s)$ represents forwarding s with the TM_i network whose parameter is θ_{T_i}. $f(\theta_{D_i}; s)$ represents the same operation performed with the student network whose parameter is θ_{D_i}. l_{T_i} and l_{D_i} are the forwarded logits value, and $s \in S$ is the sample data randomly chosen from the proxy dataset.

At this time, the purpose of the DM_i network is to make l_{D_i} and the teacher network's l_{T_i} as similar as possible. Therefore, the optimization objectives are:

$$\min_{\theta_{D_i}} = E_{s \in S}\left[Div(\sigma(\frac{l_{T_i}}{T}), \sigma(\frac{l_{D_i}}{T})\right], \tag{5}$$

where $E_{s \in S}[\bullet]$ is the expectation about the dataset S. $Div(a, b)$ measures the difference between a and b, usually Kullback-Leibler (KL) [41] or mean-square error (MSE). σ is the $softmax$ function, which is used to project the logits of the sum into the probability space; T is the distillation temperature [36], and the hard label with higher confidence in TM_i can be softened into a soft label that is easier to learn in DM_i. Then, we can use a training optimizer such as Adam [42], SGD, etc. to iteratively optimize the formula (5).

C. Self-Driving Learning Strategy Based on HCI

The traffic features of IoT devices will change dynamically according to firmware updates and user needs, so the test set features should also be dynamically adjusted to meet this requirement. The SL-based detection framework we designed enables a small number of features of a single device on the edge node to be aggregated through SL. The sufficient number of feature samples lays a good foundation for the accuracy of dynamic learning. On this basis, we propose a two-stage release strategy for the detection results. We innovatively use HCI to perform data annotation to achieve self-driving learning after deployment.

1) Network Traffic Release Strategy for HCI: As shown in Fig. 6, packet segments that passed the detection process flow generally to the target device or server, while for packet segments that failed to pass the detection process, the gateway temporarily intercepts the packet segments and sends an encrypted authentication message to the user device. Following the work of Braeken et al. [43] and Liu et al. [44], we give a brief description of the authentication process in the red box of Fig. 6. Upon receiving the request, the user device decrypts the identity parameters of the gateway to confirm its legitimacy. Then, the

Authorized licensed use limited to: King Mongkut's University of Technology Thonburi. Downloaded on October 13,2026 at 10:16:10 UTC from IEEE Xplore. Restrictions apply.

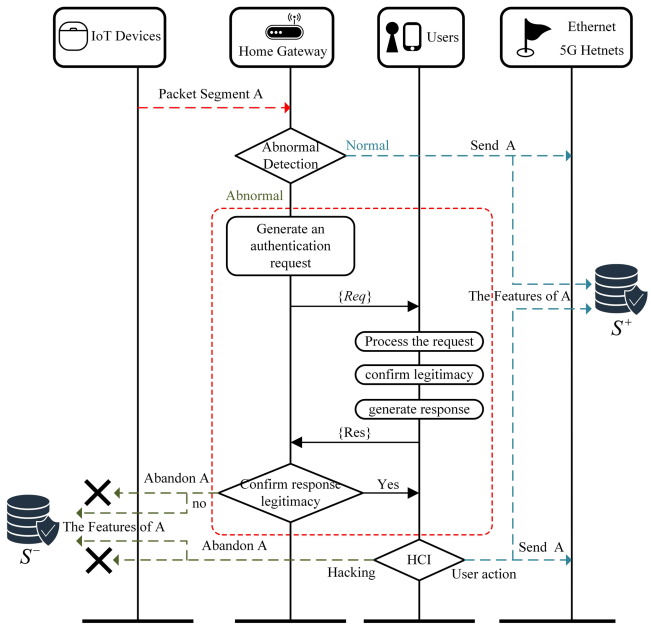


Fig. 6. Data packet forwarding and HCI process with authentication in abnormal behavior detection model.

identity parameters are transformed into a response and sent back to the gateway using symmetric or asymmetric encryption algorithms based on a pre-shared key. The gateway decrypts the response using the pre-shared key and performs consistency and integrity checks on the identity parameters to ensure the legitimacy of the device participating in data annotation. Once the authentication process is finished, the scheme sends to device a pop-up. It determines whether it is operating generally based on user feedback. The packet segment will flow to the target device or server if the user confirms. Suppose the user denies or does not answer for more than 30 seconds. In that case, the packet segment will be terminated by the gateway and deleted.

2) Self-Driving Dynamic Learning: Semi-supervised learning of this scheme stems from the continuous training of data capture and dynamic detection model. Its implementation process is shown in Algorithm 2. Specifically, according to the anomaly detection model's judgment and user interaction result, the scheme will divide the newly generated behavior characteristics data into positive and negative samples and store them in the corresponding sample dataset. Meanwhile, old data in the sample set will be removed. When energy consumption of the gateway is at the bottom level, the training layer will use the new sample set to train the local training model for related equipment. When the bandwidth occupancy is low, the original and rising parameters are aggregated together at the temporary center node of the SL network. The new parameters are then returned to the training layer of the edge node. Furthermore, the scheme performs knowledge distillation at the detection layer to reduce the complexity of the model. The distilled knowledge is retained for filing. Finally, the gateway model is deployed to the detection layer and continues to provide anomaly detection service. The service resources are sufficient when the energy consumption of the system is low. At this time, local training,

Algorithm 2: Self-Driving Learning Implementation Process.

Require: For t-th lifelong learning, positive sample set $\mathbf{S}^+ = \{s_t^+, s_{t+1}^+, s_{t+2}^+, \ldots, s_k^+\}$, negative sample set $\mathbf{S}^- = \{s_{t+1}^+, s_{t+1}^+, s_{t+1}^+, \ldots, s_k^-\}$, training model TM_t, detection model DM_t, computing consumption $C2$, energy consumption EC, bandwidth usage ratio BR

Ensure: updated training model TM_{t+1}, Updated detection model DM_{t+1}

while new positive sample feature s_x^+ appear **do**
  $s_{k+1}^+ = s_x^+$
  $\mathbf{S}^+ = \{s_t^+, s_{t+1}^+, s_{t+2}^+, \ldots, s_k^+\}.Del(s_t^+)$
  $\mathbf{S}^+ = \{s_t^+, s_{t+1}^+, s_{t+2}^+, \ldots, s_k^+\}.Add(s_{k+1}^+)$
end while
while new positive sample feature s_x^- appear **do**
  $s_{k+1}^- = s_x^-$
  $\mathbf{S}^- = \{s_t^-, s_{t+1}^-, s_{t+2}^-, \ldots, s_k^-\}.Del(s_t^-)$
  $\mathbf{S}^- = \{s_t^-, s_{t+1}^-, s_{t+2}^-, \ldots, s_k^-\}.Add(s_{k+1}^-)$
end while
if $C2$, EC are at the bottom **then**
  $\mathbf{P}^t = TM_t(\mathbf{S}^+, \mathbf{S}^-)$ ▷Get rising parameters
end if
Set the rising parameter of all swarm nodes as $\mathbf{H} = \{\mathbf{P}_1^t, \mathbf{P}_2^t, \mathbf{P}_3^t, \ldots\}$
if BR is at the bottom **then**
  Upload $\mathbf{H}$ to the aggregation node
  $TM_{t+1} = DM_t$
  $DM_{t+1}^* = SL(\mathbf{H})$ ▷Parameter aggregation using SL network
  $DM_{t+1} = KD(DM_{t+1}^*)$ ▷KD indicates knowledge distillation
end if
return DM_{t+1}, TM_{t+1}

which consumes a lot of energy, will not interfere with the anomaly detection service and the IoT system. Moreover, the existing models cannot handle traffic that may have unknown features. Consequently, the model needs to be continuously updated with new train set to dynamically improve the detection capability.

V. Experiment and Analysis

In order to verify the effectiveness of the proposed method, we collect the network traffic data of household IoT and analyze the performance, QoS, scalability, and other important indicators of malicious behavior detection.

A. Experimental Environment Setting

In this paper, we use a Huawei Wi-Fi access point (TC7102) as internal switch and internet gateway configuration. The Tcpdump running on OpenWrt_Chaos_Calmer is applied to obtain packet data. And we write bash scripts to classify and extract features. As can be seen in Fig. 7, the experimental data is from real smart household devices. In a week, we consecutively

Authorized licensed use limited to: King Mongkut's University of Technology Thonburi. Downloaded on October 13,2026 at 10:16:10 UTC from IEEE Xplore. Restrictions apply.

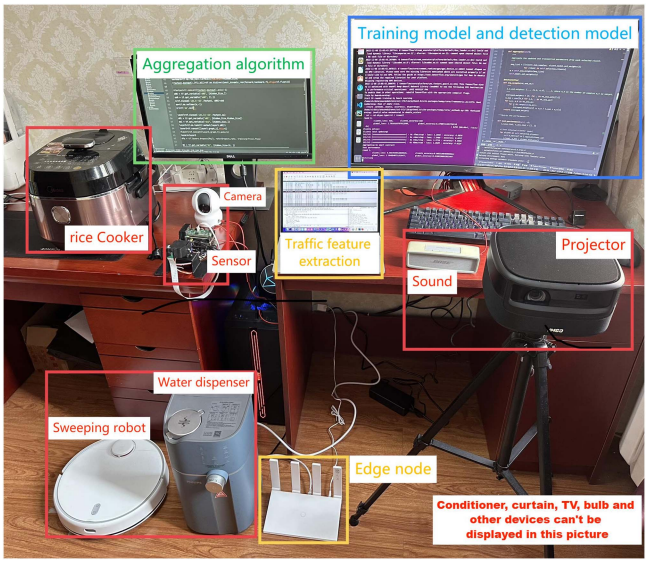


Fig. 7. Experimental deployment scenario.

TABLE I
MODELS AND QUANTITY OF THE EXPERIMENTED SMART HOUSEHOLD DEVICES

Device	Model	Quantity
Smart Camera	HUAWEI CV70	2
Smart Power Strip	MI ZNCXB01ZM	7
Smart Bulb	OPPLE Smart Ceiling Light	17
Smart Refrigerator	Midea BCD-600WKGPZM(E)	1
Smart TV	HUAWEI HD86KEPA	1
Smart Curtain	DOOYA DH6	5
Sweeping Robot	ECOVACS T10TURBO	1
Smart Projector	JMGO V20S	1
Smart Sensor	Raspberry Pi 4B	1
Smart Oven	Midea PS20C2W	1
Smart Water Dispenser	Midea KFR-35GW/N8HK1	1
Smart Sound	BOSE SoundLink Mini	1
Smart Conditioner	COLMO KFR-72LW/CE1H	1
Smart rice Cooker	Midea 50FS69-F	1
Smart Door Lock	HAUWEI AGS-S10	1
Smoke Alarm	Heiman JM-720	6
Smart Washing Machine	COLMO CLDS10E	1

collected network traffic data of 49 household IoT devices of 17 types, including smart washing machines, smart rice cookers, smart door locks, smart bulbs, smart power strips, human body sensors and so on. The specific models and specifications are shown in the Table I.

Based on these smart home devices, we constructed the experimental environment and combined these home IoT devices according to the house-wide customized smart home solutions to simulate the data flow in the natural environment. After deploying these devices and environments, we collected data according to the normal lifestyle and added specific attack methods.

B. Data Preprocessing

After completing the data collection above, we cleaned and preprocessed the collected data. Specifically, we deleted the data with incomplete samples, missing information, and time sequence isolation in the data cleaning. Then, the remaining data were preprocessed, including discrete value mapping, sample feature fusion, data segmentation, etc. The preprocessing steps are as follows.

- Discrete value mapping: We map discrete features such as port, IP, service, etc. in the traffic data to the integer space for input to the neural network, e.g., src port values ranging from $\{80, 88, 90, 8888, \ldots\}$ is transformed into $\{1, 2, 3, 4, \ldots\}$
- Sample feature fusion: Because there are multiple requests simultaneously, it is necessary to fuse and extract the simultaneous feature information to facilitate the training and detection models to find hidden knowledge. After processing, the adopted characteristic fields and their meanings are shown in the Table II.
- Data segmentation: After the traffic data is arranged in chronological order, we take out specific data segment and splice them into matrices to be applied in fitting the temporal relationship. Instead of using a fixed time window, the data are processed by combining multiple packets at fixed time intervals to form a training sample based on the acquisition order. For example, the characteristic vector of $t = 1$ is $[0, 1, 2, 3]$, the characteristic vector of $t = 2$ is $[2, 2, 2, 3]$, and the characteristic vector of $t = 3$ is [0,0,0,0]. If the time step is 2, then the sample after splicing is $\begin{bmatrix} 0 & 1 & 2 & 3 \\ 2 & 2 & 2 & 3 \end{bmatrix}$ and $\begin{bmatrix} 2 & 2 & 2 & 3 \\ 0 & 0 & 0 & 0 \end{bmatrix}$. The sampling time is accurate to ms to improve the feature analysis of the model.

After data cleaning and preprocessing, a total number of 316564 samples were obtained, including five types of attack: injection, ransomware, scanning, ddos, and dos. In addition, there are two normal types: normal and password. To make the training sample more balanced, we reduced the proportion of normal and password in the data preprocessing. The number of samples in the data set is shown in Table II. Traffic features are processed based on the underlying information of the packet, and are usually chosen to be clearly distinguishable between normal and malicious traffic. Based on this principle, we extract specific data features such as timestamp, source IP, source port, destination IP, destination port, duration, and size.

We evaluate the performance of ADONIS through the traffic dataset based on 17 devices with 6 attack methods. The model's fitting and generalization capabilities are examined through a rigorous machine learning process of training, validation, and testing. In addition, we experiment in home IoT where newly smart devices and attacks are deployed to further demonstrate that the model has excellent generalization ability.

C. Results and Discussion

1) Share Parameters and Benchmark Algorithms: We simulated a SL scenario involving 20 home gateways (clients) based on the collected data. It runs on a Lenovo Saver Blade 7000 k with an Intel core i7-12700f processor, NVIDIA RTX3070, 16G DDR5 RAM and 512 G SSD. The operating system is Ubuntu 20.04, and the training framework is TensorFlow 2.3.0. In the experiment, the probability of the client being activated and

Authorized licensed use limited to: King Mongkut's University of Technology Thonburi. Downloaded on October 13,2026 at 10:16:10 UTC from IEEE Xplore. Restrictions apply.

TABLE II
THE TYPES OF ATTACK AND NUMBER OF SAMPLES IN THE CLEANED DATASET

Type of Attack	Number of Samples	Type of Attack	Number of Samples	Type of Attack	Number of Samples
injection	63911	scanning	50520	dos	6641
normal	52991	password	38757	ddos	49999
ransomware	53745	-	-	-	-

TABLE III
COMPREHENSIVE COMPARISON BETWEEN THE DESIGNED ALGORITHM AND THE BENCHMARK ALGORITHM

Algorithm	Accuracy		Parameter scale		Inference Time (ms)	Average traffic (MB/r)	Ratio of Inferencing Time	Communication Compression Ratio
TM	67.6%		1,434,368		56.23	0	81.2%	0%
DM	44.4%		68,832		8.35	0	12.1%	0%
SL-TM	89.4%		1,434,368		65.75	5.47	94.9%	21.0%
SL-DM	66.3%		68,832		9.76	0.26	14.1%	1.0%
Centralized-TM	92.2%		1,434,368		69.29	25.99	-	-
Centralized-DM	66.8%		68,832		7.71	20.52	11.1%	78.9%
Ours	Training	Detection	Training	Detection	8.22	5.44	11.9%	20.9%
	89.6%	82.2%	1,434,368	68,832				

participating in one round of training is 0.2, the local training epochs of each client is 5, a total of 55 rounds are carried out, and the input data sequence length is 32. Each client has only the local traffic generated by itself, and the data from all clients together constitute the complete dataset. We divide the raw data into training set, development set and test set according to $7:2:1$ for model evaluation. When the client takes local training, the learning rate is $5e^{-4}$, and the batch size is 16. KL Divergence is used as the loss of knowledge distillation, enabling students to learn teachers' classification methods instead of simple imitation. Distillation temperature $T = 20$, and distillation learning rate is $1e^{-4}$.

In order to verify the effectiveness of the proposed method, we select six types of algorithms as the benchmark of behavior recognition, namely:

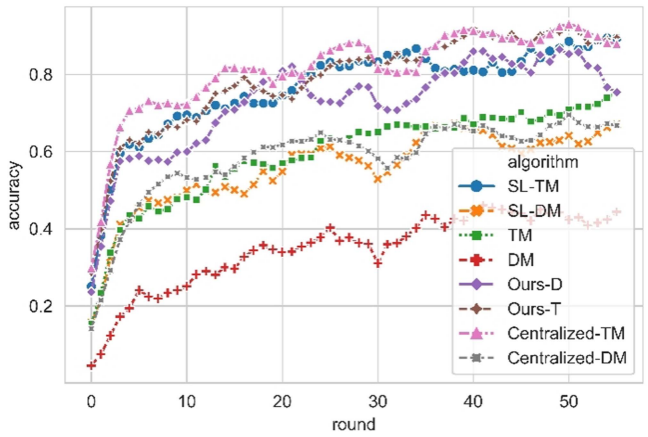
- TM: each home gateway independently trains the TM model based on local data, and the TM model screens the behavior of nodes;
- DM: each home gateway independently trains the DM model based on local data, and the DM model screens the behavior of nodes;
- SL-TM: All home gateways use SL to train the TM model cooperatively and use the shared TM model based on ascending parameter aggregation to screen the behavior of nodes;
- SL-DM: All home gateways use SL to train the DM model collaboratively and use the shared DM model to screen the behavior of nodes;
- Centralized-TM: centralize all data to a central node to carry out TM model training, and then share TM model for screening;
- Centralized-DM: centralize all data to a central node to carry out DM model training, and then share DM model for screening.

These six algorithms can test the performance of typical standard methods in the set scenario, such as SL, Isolated Training, Centralized Training, Small Model (DM), and Large Model (TM), to provide a fair and comprehensive comparison for the proposed methods. Because the centralized approach requires all user data to be uploaded to the server, exposing the user's privacy, we use it only as a baseline for comparison, not as a substitute for the proposed scheme.

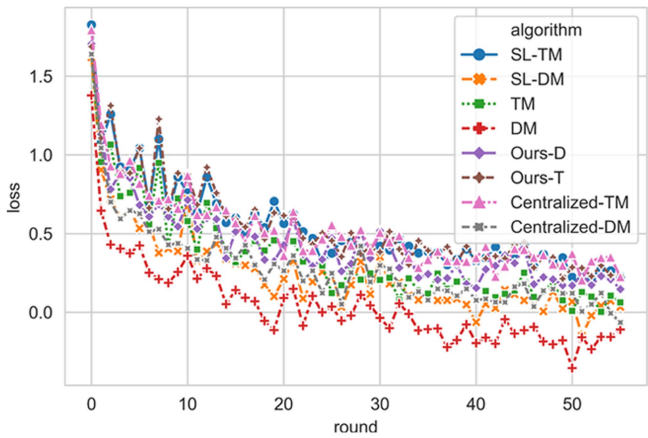
2) Comprehensive Comparison: Table III shows a comprehensive comparison between the algorithm we designed ourselves and the benchmark algorithm. Among them, the accuracy rate is the proportion of accurate prediction results on the test set after the model is trained according to different methods. The parameter scale is the number of parameters of the models used in different benchmark algorithms. Since TM, SL-TM, and Centralized-TM use the TM model, their parameter scale is 1,434,368. The DM model is similar. Inference time is required for inferencing on the same sample data size after model training. This value is obtained by exporting the trained model and using TensorFlow Lite to conduct an inference test on a Raspberry Pi 4B device. The average communication traffic is the data required for each node simulated in the experiment to transmit local information, including data that the model transfers in SL and local data uploads in Centralized. Inference time and communication compression ratios are the comparative results calculated according to inference time and average communication volume.

According to the experimental results in Table III, Centralized-TM performs best when only comparing the accuracy. However, the centralized method requires users to expose all local data, which poses a significant threat to user privacy. In addition, the centralized method also lays heavy burden on communication and computing resources, especially with the increasing size of edge nodes; the data storage and processing capacity of the central node are also facing enormous challenges. SL can solve the above problems very well and achieve similar results. In addition, after applying knowledge distillation, our scheme reduces the parameter size of the detection process from 1,434,368 to 68,832 compared to the original SL-TM and TM methods, and achieves an accuracy close to that of TM. This

Authorized licensed use limited to: King Mongkut's University of Technology Thonburi. Downloaded on October 13,2026 at 10:16:10 UTC from IEEE Xplore. Restrictions apply.



(a) Accuracy change curve.



(b) Loss change curve.

Fig. 8. Golbac accuracy change in the training.

allows satisfactory results to be achieved with fewer parameters, significantly cutting storage costs. And the accuracy of SL-TM is only 2.6% lower than that of Centralized-TM, but the cost of communication can be reduced to 21.0%, greatly reducing the energy consumption. With the increase in the number of nodes, this optimization will become more prominent. However, the use of SL alone cannot meet the needs of model accuracy and fast inference. A very small-scale SL-DM can only achieve an accuracy of 66.3%, leading to many misjudgments. In addition, SL-TM requires a lot of inferencing time, which can lead to a bad user experience. In contrast, our proposed training-detection decoupling method can guarantee a relatively satisfactory detection effect (increased by 15.9% compare to SL-DM) while reducing the device inference time to 11.9% (reduced by 8.4 times compare to SL-TM), which decreases the communication delay.

3) Effectiveness of Training-Detection Decoupling: In order to compare the effectiveness of the training-detection decoupling mechanism in the proposed method, we demonstrated the learning effects of the proposed methods and the benchmark algorithm in Fig. 8 below. Fig. 8(a) is the accuracy change curve, and Fig. 8(b) is the loss change curve. Ours-T and Ours-D, respectively, correspond to the accuracy of the proposed method's training and detection layers. The TM and DM using only local data can be regarded as the lower limit of the model, and the TM and DM using the centralized method can be regarded as the upper limit of the model.

Although the proposed method only uses the model structure of DM in the detection layer, it can achieve an effect that exceeds the upper limit of DM. However, only using the DM model for SL can achieve an effect close to Centralized-DM. This is because the DM model is weak, and the ability to learn knowledge directly from the original data is not strong. Therefore, using the DM model alone can reduce the inference delay, but it significantly impacts the detection effect. The proposed method can use the TM model, which is much stronger, extracting the knowledge from the original data, and transferring it to the TM model in a more understandable way, to achieve satisfactory results. In addition, in the training layer, our model and SL-TM use the same strategy and achieve similar performance. This enables the proposed method to enjoy the powerful performance of large models and the fast inference speed of small models simultaneously to achieve more satisfactory results.

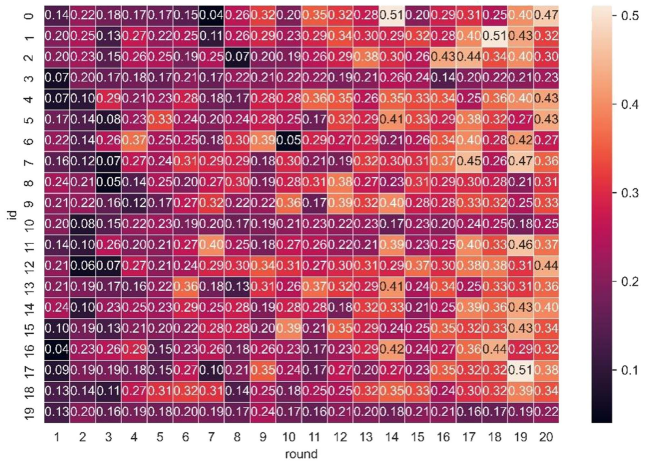
4) Convergence of Clients: Fig. 9 shows the detection accuracy of all clients in the system during the training. The lighter the color, the higher the accuracy. Because both SL-DM and SL-TM adopt the SL method, and both DM and TM adopt the localization method, the two methods lead to similar convergence of clients. Therefore, we choose SL-DM and DM to represent SL and localization-like methods, respectively.

As Fig. 9 depicts, there are stubborn clients (ID = 2, 3, 10, 19, etc.) in the localization-like method training, which results from data insufficiency. For example, most home gateways may never have been attacked, so there is data in their local devices that can be used to identify malicious behavior. However, as we discussed above, collecting their data to a central node is not acceptable as that could lead to user privacy disclosure and the cost of network traffic. Compared with the method, the SL method becomes an inevitable choice. In addition, in Fig. 9(b) and (c), although the two methods show similar convergence, the proposed method leads to higher detection accuracy.

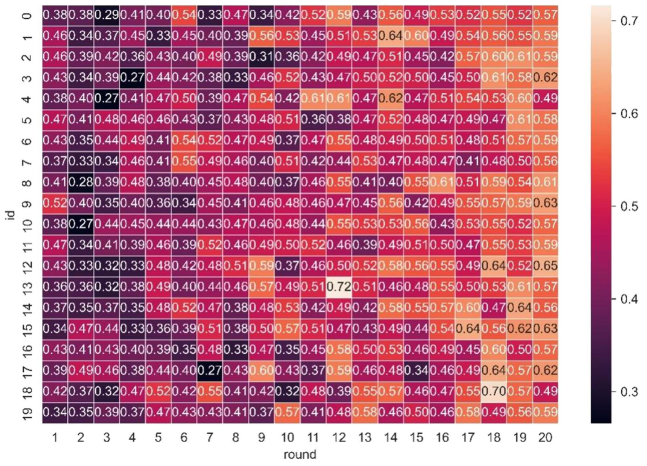
5) Effectiveness of Knowledge Distillation: In the proposed method, we adopt knowledge distillation to distill the knowledge from TM to DM to improve the DM performance. However, if the performance of DM is good enough, can it achieve similar results without knowledge distillation. To answer this question, we compare the effects of direct training of six types of DM models and the effects of TM-assisted distillation models with the same data scale and computational consumption. These six DM models are one-dimensional CNN (1D-CNN), single-layered LSTM (light-LSTM), stacked LSTM (Stack-LSTM), deeper DNN (D-DNN), wider DNN (W-DNN) and original DM model (DNN) respectively.[2]

[2]The DM models and their structural parameters are as follows
- 1D-CNN: FC(32) → relu → Conv1D(5, 3) → FC(6) → softmax.
- Light-LSTM: FC(32) → relu → LSTM(32) → FC(6) → softmax.
- Stack-LSTM: FC(32) → relu → LSTM(16) → LSTM(16) → FC(6) → softmax.
- D-DNN: FC(32) → relu → FC(16) → relu → FC(6) → softmax.
- W-DNN: FC(512) → relu → FC(6) → softmax.
- DNN: Consistent with the structure described in Section IV-B3.

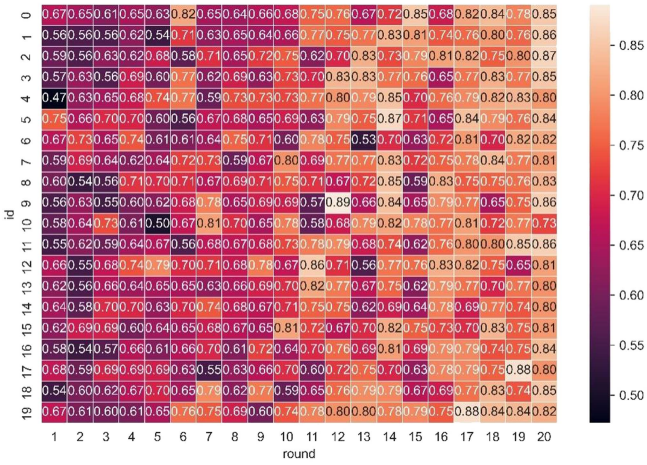
Authorized licensed use limited to: King Mongkut's University of Technology Thonburi. Downloaded on October 13,2026 at 10:16:10 UTC from IEEE Xplore. Restrictions apply.



(a) Localization-like method (DM).



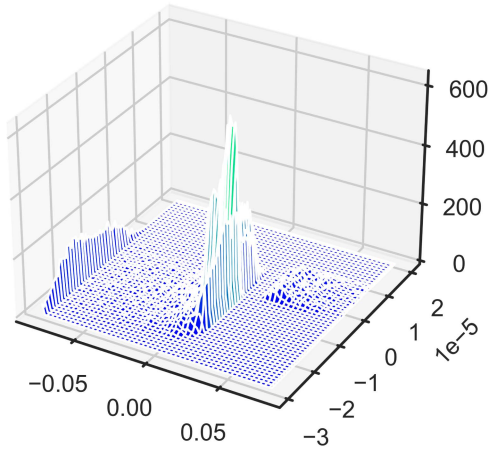
(b) SL like method (SL-DM).



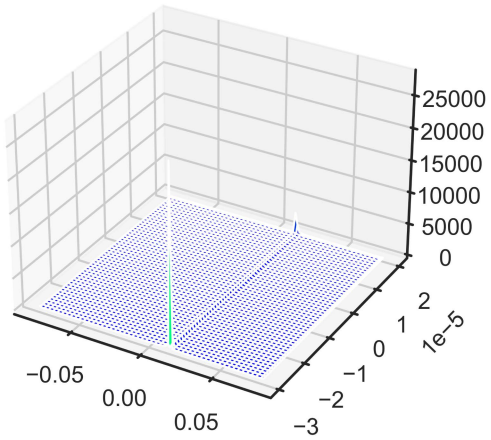
(c) Training-detection decoupled ADONIS (Ours).

Fig. 9. Distribution of client accuracy during the training.

As Table IV shows, whether it is a one-dimensional convolutional neural network, light LSTM, stack LSTM, or wider DNN, the performance of the direct training DM model is consistently worse than that of the TM-assisted distillation models. The gap is more noticeable when the structure of the DM model is simple,



(a) Weight-gradient distribution after direct training (clip $(3e^{-5}, 2e^{-5})$).



(b) Weight-gradient distribution after distillation (clip $(1e^{-7}, 1e^{-7})$).

Fig. 10. Comparison of weight-gradient values between direct training and TM assisted knowledge learning.

such as DNN. The difference between the two begins to narrow as the model structure becomes more complex, such as CNN, LSTM, and other structures. This is due to the improvement of the representation of the DM model. However, when the result of the DM model is too complex, it will bring an inference delay that cannot be ignored. Therefore, knowledge distillation can effectively improve the performance of a relatively more minor DM model. It should be noted that the proposed ADONIS is model-free, which focuses on the framework rather than a specific model structure. Therefore, no matter what neural networks, similar results can be achieved.

Fig. 10 visualizes all parameter weights for the DM model (DNN) with knowledge distillation and the DM model (DNN) with direct training. They respectively show all the learning parameters in the DM model and their gradient values and statistical rules. The darker the color, the higher the frequency of the weight value. In order to facilitate visualization, the gradient values are clipped to highlight the data distribution.

As seen from Fig. 10, the difference in weight distribution of the DM model after distillation is more evident than that of the

Authorized licensed use limited to: King Mongkut's University of Technology Thonburi. Downloaded on October 13,2026 at 10:16:10 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
PERFORMANCE IMPROVEMENT OF DM MODEL THROUGH KNOWLEDGE DISTILLATION

	1D-CNN	Light-LSTM	Stack-LSTM	D-DNN	W-DNN	DNN
TM Assisted	81.1%	83.7%	85.6%	82.6%	78.5%	82.2%
Direct Training	71.7%	75.9%	80.4%	49.2%	45.1%	44.4%

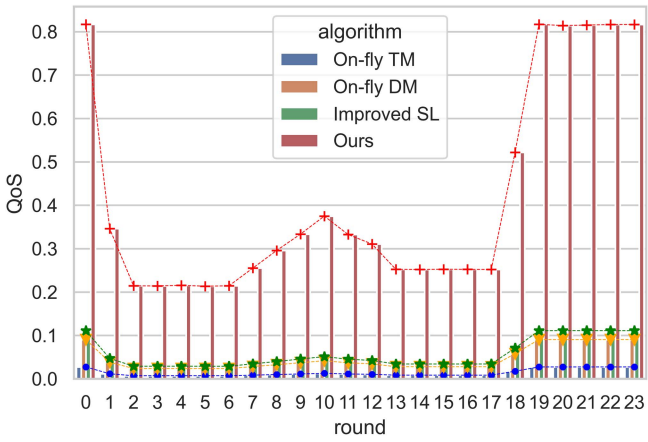


Fig. 11. QoS change of device used in 1 day.

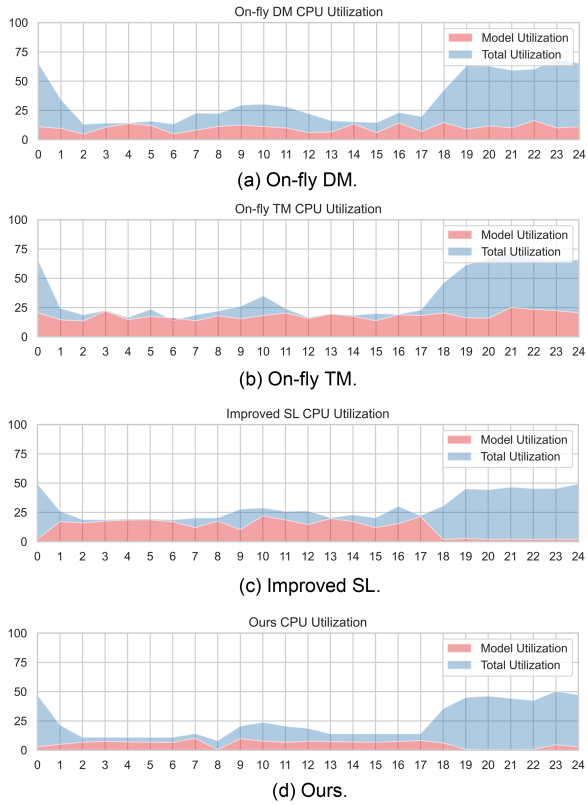


Fig. 12. Proportion of malicious behavior detection in CPU utilization.

direct training. This shows that the DM model can better find the critical information in the input data after the distillation from TM knowledge and better explore the relationship among them. In addition, there is also a vast difference between the two distributions, which shows that knowledge distillation effectively affects the training director of the DM model and helps to discover the learned information.

In addition, knowledge distillation is one of the most widely used model compression techniques. Compared with methods such as model pruning, the distilled model does not need to adopt the same structure as the original model. For example, knowledge in CNN can be distilled into MLP, while model pruning cannot. This method enables the ADONIS scheme to work in a model-free mode, while model pruning cannot. In addition, the two do not conflict and can coexist. And after model pruning, fine tune needs to be performed again, which requires the support of original data. However, user data cannot be shared in IoT anomaly detection scenarios, which makes model pruning infeasible. And knowledge distillation that only shares parameters is very applicable.

6) User Experience Comparison: We compare the QoS (Quality of Service) between the proposed method and the traditional method. To facilitate the analysis, we set the waiting time of users as the main indicator of QoS and define $QoS = \frac{1}{t_w} \cdot p_u \cdot acc$ with. Among these, t_w is the time to wait for the gateway result, p_u is the probability of using the device (gateway) at the current time, and acc is inference accuracy. p_u is set according to the frequency of using the device every 24 hours in the lab environment. This means that when the frequency is high, the shorter the inference time and the higher the accuracy, the greater the QoS.

Fig. 11 shows the user experience of using the device in one day. Among them, on-fly DM and on-fly TM adopt the traditional strategy of training and inferring simultaneously. Improved SL follows our strategy to modify the training time to night (when the computational force is at the valley) but adopts the TM model. Ours adopts the training-detection decoupling framework when computational power is at the valley. As Fig. 11 shows, the QoS of our method is significantly better than others as we create a smooth user experience. This results from our adjustment of the training process that consumes the most computational resources to the period when users rarely use it. We use the lightweight DM model for inference instead of the TM model.

Fig. 12 visualizes the Proportion of the malicious behavior detection in the computational consumption, showing that our framework can adapt to the computational situation and adjust the service strategy. As the DM model is lightweight, it can shorten and control the inference time. By contrast, TM's inference time is much longer, and there is more significant uncertainty. It can be concluded that the framework we proposed can provide users with more rapid and stable detection services, which effectively improves the user experience.

Authorized licensed use limited to: King Mongkut's University of Technology Thonburi. Downloaded on October 13,2026 at 10:16:10 UTC from IEEE Xplore. Restrictions apply.

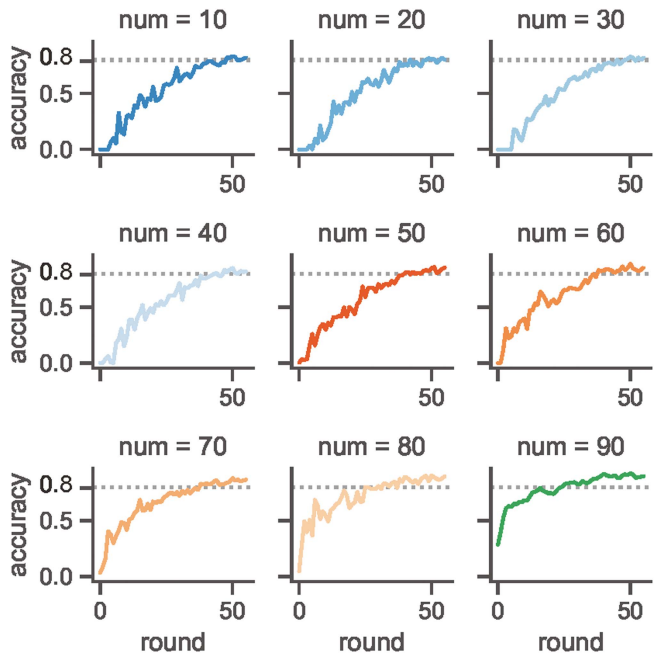
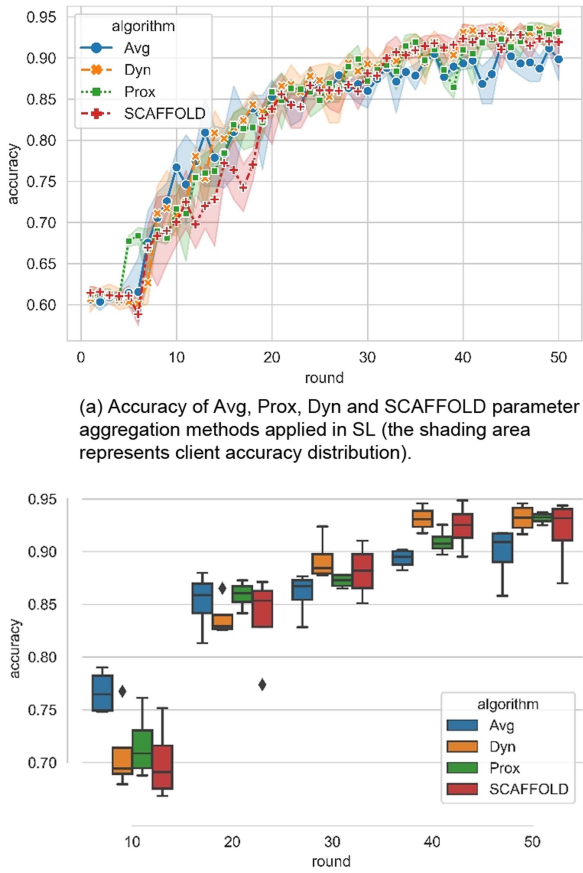


Fig. 13. System extensibility. Change the accuracy rate when the number of clients changes. In the figure, the gray auxiliary line marks that accuracy = 0.8.

TABLE V
ABNORMAL DETECTION EFFECTIVENESS UNDER DIFFERENT DATASET SCALE

Dataset Size	DM	SL-DM	C-DM	Ours
1k	47.3%	68.0%	69.5%	83.1%
10k	46.8%	67.4%	68.4%	82.6%
100k	43.7%	65.8%	66.1%	80.0%
500k	42.9%	65.1%	65.8%	79.6%
1000k	41.5%	64.5%	65.9%	78.4%

And C-DM refers to centralized-DM.



(a) Accuracy of Avg, Prox, Dyn and SCAFFOLD parameter aggregation methods applied in SL (the shading area represents client accuracy distribution).

(b) Convergence effect of Avg, Prox, Dyn and SCAFFOLD para meter aggregation methods applied in SL.

Fig. 14. Comparison of system training effects with different aggregation algorithms.

7) Extensibility of the System: Fig. 12 shows the impact of changes in the node number on overall system performance. We increase the number of edge gateways from 10 to 90 and observe the change in the accuracy of malicious behavior detection when the number of gateways changes.

As Fig. 13 shows, with the number of nodes grow, the system needs more time to converge the accuracy of malicious behavior detection by increasing the node number. This is because when the node number increases, the size of data and the number of malicious behavior samples stored in each node decreases, making it more challenging to converge data and knowledge. However, the system can still converge to a more satisfactory result and displays good extensibility.

8) Extensibility of Dataset Scale: In the previous experiment, we evaluated by 316,564 samples among 7 different behavior types. However, another question is, as the dataset keeps increasing, how will the performance of the system change? This is an inevitable problem in online running systems. Therefore, based on the above 316,564 samples, we adjusted the above dataset to five different data scales, such as 1 k, 10 k, 100 k, 500 k and 1000 k, by repeated random sampling. The experimental results are shown in Table V.

According to the results shown in Table V, with the increase of dataset size, the accuracy of abnormal behavior detection of all algorithms decreases. This is because, with the increase of dataset scale, the sample size of nontrivial abnormal behaviors in the original data set increases, but the overall proportion decreases and the learning effect decreases. However, no matter how the sample size changes, our method can always achieve better results than the other methods with the same resource overhead.

9) Impact of the SL Aggregation Algorithm: In the method proposed in this paper, we follow SL [15] to aggregate ascending parameters. To further analyze the impact of different aggregation algorithms on the performance of the ADONIS scheme, we deploy the SL versions of three aggregation methods, Prox [45], SCAFFOLD [46], and Dyn [47]. The impact of these aggregation methods on accuracy is shown in Fig. 14.

It can be seen that these three types of aggregation methods can improve the overall performance of the system to varying degrees. Overall, the aggregation algorithm has a significant impact on system performance. Therefore, in practice, we can adopt proximity terms and control variables to improve system performance. Similar to the discussion of Warnat-Herresthal et al. [15], our framework can adopt any emerging aggregation methods that

Authorized licensed use limited to: King Mongkut's University of Technology Thonburi. Downloaded on October 13,2026 at 10:16:10 UTC from IEEE Xplore. Restrictions apply.

perform well in FL. This provides a huge performance improvement space for the proposed scheme. However, compared with FL, ADONIS has the advantage of adopting SL framework. SL adopts a decentralized design based on blockchain technology (smart contract), and the models uploaded by all nodes are traceable, which ensures the safety and transparency of the learning process. In addition, due to the decentralized structure, the SL-based ADONIS scheme is better at discovering abnormal behaviors of nodes, and thus has stronger robustness against attack methods such as model poisoning, which shows more stable performance than FL. Related conclusions can be found in the work of Bagdasaryan et al. [48].

D. Theoretical Analysis of Combined Swarm Learning

The ADONIS based on SL has several advantages when applied to abnormal behavior detection. First, SL, an advanced machine learning framework with the virtue of passing only parameters without data, can provide robust privacy protection. The ADONIS deploys each IoT node as a local model and enables communication links among the nodes through a swarm network. The local model only needs to be trained based on the traffic data in IoT, and the results can be shared with other nodes without sharing training data. This mode greatly increases data security and can avoid the risk of privacy leakage caused by the proliferation of user data. Second, SL is implemented based on blockchain technology, which is open and inclusive, and has strong scalability. IoT nodes can access the SL system as long as they have been authenticated to confirm their legitimacy. The ADONIS treat it as an edge model and receive the trained local parameters, and give it the authority to use the integrated model. Third, compared with traditional methods, the model complexity of SL is lower, and it is an efficient learning mode. SL utilizes large-scale feature extraction of distributed machine learning to avoid the pressure of computing and storage brought about by centralized data analysis. ADONIS distributes the training process to each edge node, and completes the traffic analysis by fusing the local model parameters together through basic calculations, which greatly reduces the resource load pressure and has faster training and detection efficiency. Finally, SL shows excellent detection performance. Its mechanism is equivalent to first cutting large-scale data for separate training and then collecting parameters for unified integration. In essence, it is still model learning based on all user data. This method preserves all IoT traffic characteristics, and when combined with an appropriate neural network model, it can ensure high accuracy of device anomaly detection.

VI. Conclusion

Aiming at the problem of abnormal behavior detection in IoT devices, this paper proposes an interactive dynamic learning behavior detection framework, ADONIS. It focuses on the continuous improvement of user experience and detection performance by designing an advanced framework that perfectly integrates training and detection decoupling, HCI-based self-driven learning and SL, and knowledge distillation. Specifically, we devise SL-based abnormal behavior detection model to enable the fusion of multi-device data by sharing only the parameters without transmitting the raw data. This improves the accuracy of the model and avoids user privacy leakage. And the decentralized structure of SL is not constrained by single node damage, strongly reinforcing the stability of the detection service. Meanwhile, we design full data training and knowledge distillation simplification methods to compress the high accuracy full model into a lightweight detection model, and decouple training and detection to optimize the latency and energy consumption metrics. We also innovatively develop HCI mechanisms based on artificial+intelligence to dynamically train the model in a user-insensitive mode with continuously updated data samples to maintain sensitivity to unknown behaviors. The superiority of ADONIS is validated in a typical home IoT, achieving excellent results of detection accuracy of 82.2%, average latency of 8.22 ms, and model complexity simplified to 15.9%, which meets the demand of lightweight and fast response. In addition, compared with the approach using SL alone, our proposed scheme shortens the detection delay by a factor of 8.4 and decreases the accuracy by no more than 7.4%, demonstrating the effectiveness of combining decoupling and knowledge distillation.

In future work, we intend to investigate the following aspects:

- *Further promotion and improvement:* The method proposed in this paper can be applied to ubiquitous IoT and mobile computing scenarios such as smart homes, smart city, and the Internet of Vehicles. It can be extended to more meaningful scenarios by perfecting the work and designing a prototype system based on existing work.
- *Higher communication efficiency:* In the framework proposed in this paper, the collaborative design of SL mainly follows the method of Warnat-Herresthal et al. [15], which has excellent stability. However, a complete transmission intelligence model will still bring a large communication burden. This can be further compressed by applying gradient quantization, low-rank decomposition, and random sampling.
- *More appropriate parameter aggregation methods:* In the parameter aggregation section, we use the Avg class method for aggregation. This has good applicability for IID data scenarios. However, when the data distribution characteristics are generalized to the NIID scenario (which is more common in reality), the parameter aggregation method needs targeted improvement. This can be achieved by adding regular aggregation terms [45], modifying the aggregation paradigm [49], etc.

References

[1] X. Liang and Y. Kim, "A survey on security attacks and solutions in the IoT network," in *Proc. IEEE 11th Annu. Comput. Commun. Workshop Conf.*, 2021, pp. 0853–0859.

[2] H. Li et al., "Understanding and detecting remote infection on linux-based IoT devices," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, 2022, pp. 873–887.

[3] L. Cui et al., "Security and privacy-enhanced federated learning for anomaly detection in IoT infrastructures," *IEEE Trans. Ind. Inform.*, vol. 18, no. 5, pp. 3492–3500, May 2022.

Authorized licensed use limited to: King Mongkut's University of Technology Thonburi. Downloaded on October 13,2026 at 10:16:10 UTC from IEEE Xplore. Restrictions apply.

[4] C. Zhang, X. Costa-Pérez, and P. Patras, "Adversarial attacks against deep learning-based network intrusion detection systems and defense mechanisms," *IEEE/ACM Trans. Netw.*, vol. 30, no. 3, pp. 1294–1311, Jun. 2022.

[5] C. Kolias, G. Kambourakis, A. Stavrou, and J. Voas, "DDoS in the IoT: Mirai and other botnets," *Computer*, vol. 50, no. 7, pp. 80–84, 2017.

[6] H. Liang, J. Wu, S. Mumtaz, J. Li, X. Lin, and M. Wen, "MBID: Micro-blockchain-based geographical dynamic intrusion detection for V2X," *IEEE Commun. Mag.*, vol. 57, no. 10, pp. 77–83, Oct. 2019.

[7] Y. Wang, X. Du, Z. Lu, Q. Duan, and J. Wu, "Improved LSTM-based time-series anomaly detection in rail transit operation environments," *IEEE Trans. Ind. Inform.*, vol. 18, no. 12, pp. 9027–9036, Dec. 2022.

[8] H. HaddadPajouh, A. Dehghantanha, R. M. Parizi, M. Aledhari, and H. Karimipour, "A survey on Internet of Things security: Requirements, challenges, and solutions," *Internet Things*, vol. 14, 2021, Art. no. 100129.

[9] M. Baza, A. Salazar, M. Mahmoud, M. Abdallah, and K. Akkaya, "On sharing models instead of data using mimic learning for smart health applications," in *Proc. IEEE Int. Conf. Inform., IoT, Enabling Technol.*, 2020, pp. 231–236.

[10] V. Mothukuri, P. Khare, R. M. Parizi, S. Pouriyeh, A. Dehghantanha, and G. Srivastava, "Federated-learning-based anomaly detection for IoT security attacks," *IEEE Internet Things J.*, vol. 9, no. 4, pp. 2545–2554, Feb. 2022.

[11] X. Wang et al., "Toward accurate anomaly detection in industrial Internet of Things using hierarchical federated learning," *IEEE Internet Things J.*, vol. 9, no. 10, pp. 7110–7119, May 2022.

[12] S. Arisdakessian, O. A. Wahab, A. Mourad, H. Otrok, and M. Guizani, "A survey on IoT intrusion detection: Federated learning, game theory, social psychology and explainable ai as future directions," *IEEE Internet Things J.*, vol. 10, no. 5, pp. 4059–4092, Mar. 2023.

[13] Z. Zhang et al., "LSFL: A lightweight and secure federated learning scheme for edge computing," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 365–379, 2023.

[14] Z. Wang, Y.-F. Liu, and L. Liu, "Covariance-based joint device activity and delay detection in asynchronous mMTC," *IEEE Signal Process. Lett.*, vol. 29, pp. 538–542, 2022.

[15] S. Warnat-Herresthal et al., "Swarm learning for decentralized and confidential clinical machine learning," *Nature*, vol. 594, no. 7862, pp. 265–270, 2021.

[16] S. M. Alamouti, F. Arjomandi, and M. Burger, "Hybrid edge cloud: A pragmatic approach for decentralized cloud computing," *IEEE Commun. Mag.*, vol. 60, no. 9, pp. 16–29, Sep. 2022.

[17] R. Doshi, N. Apthorpe, and N. Feamster, "Machine learning DDoS detection for consumer Internet of Things devices," in *Proc. IEEE Secur. Privacy Workshops*, 2018, pp. 29–35.

[18] H. Tahaei, F. Afifi, A. Asemi, F. Zaki, and N. B. Anuar, "The rise of traffic classification in iot networks: A survey," *J. Netw. Comput. Appl.*, vol. 154, 2020, Art. no. 102538.

[19] Y. Meidan et al., "N-BaIoT-network-based detection of iot botnet attacks using deep autoencoders," *IEEE Pervasive Comput.*, vol. 17, no. 3, pp. 12–22, Third Quarter, 2018.

[20] L. Lyu, J. Jin, S. Rajasegarar, X. He, and M. Palaniswami, "Fog-empowered anomaly detection in IoT using hyperellipsoidal clustering," *IEEE Internet Things J.*, vol. 4, no. 5, pp. 1174–1184, Oct. 2017.

[21] A. Sivanathan et al., "Classifying IoT devices in smart environments using network traffic characteristics," *IEEE Trans. Mobile Comput.*, vol. 18, no. 8, pp. 1745–1759, Aug. 2019.

[22] S. Rezvy, Y. Luo, M. Petridis, A. Lasebae, and T. Zebin, "An efficient deep learning model for intrusion classification and prediction in 5G and IoT networks," in *Proc. 53rd Annu. Conf. Inf. Sci. Syst.*, 2019, pp. 1–6.

[23] L. Xiao, Y. Li, X. Huang, and X. Du, "Cloud-based malware detection game for mobile devices with offloading," *IEEE Trans. Mobile Comput.*, vol. 16, no. 10, pp. 2742–2750, Oct. 2017.

[24] B. Weinger, J. Kim, A. Sim, M. Nakashima, N. Moustafa, and K. J. Wu, "Enhancing IoT anomaly detection performance for federated learning," *Digit. Commun. Netw.*, vol. 8, no. 3, pp. 314–323, 2022.

[25] T. D. Nguyen, S. Marchal, M. Miettinen, H. Fereidooni, N. Asokan, and A. Sadeghi, "Dïot: A federated self-learning anomaly detection system for IoT," in *Proc. IEEE 39th Int. Conf. Distrib. Comput. Syst.*, Dallas, TX, USA, 2019, pp. 756–767. [Online]. Available: https://doi.org/10.1109/ICDCS.2019.00080

[26] Y. Zhao, J. Chen, D. Wu, J. Teng, and S. Yu, "Multi-task network anomaly detection using federated learning," in *Proc. 10th Int. Symp. Inf. Commun. Technol.*, 2019, pp. 273–279.

[27] Y. Liu, N. Kumar, Z. Xiong, W. Y. B. Lim, J. Kang, and D. Niyato, "Communication-efficient federated learning for anomaly detection in industrial Internet of Things," in *Proc. IEEE Glob. Commun. Conf.*, 2020, pp. 1–6.

[28] Y. Liu et al., "Deep anomaly detection for time-series data in industrial IoT: A communication-efficient on-device federated learning approach," *IEEE Internet Things J.*, vol. 8, no. 8, pp. 6348–6358, Apr. 2021.

[29] T. V. Phan and T. Bauschert, "DeepAir: Deep reinforcement learning for adaptive intrusion response in software-defined networks," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 3, pp. 2207–2218, Sep. 2022.

[30] B. Ghimire and D. B. Rawat, "Recent advances on federated learning for cybersecurity and cybersecurity for federated learning for Internet of Things," *IEEE Internet Things J.*, vol. 9, no. 11, pp. 8229–8249, Jun. 2022.

[31] J. Wang, H. Han, H. Li, S. He, P. K. Sharma, and L. Chen, "Multiple strategies differential privacy on sparse tensor factorization for network traffic analysis in 5G," *IEEE Trans. Ind. Inform.*, vol. 18, no. 3, pp. 1939–1948, Mar. 2021.

[32] M. E. Z. N. Kambar, A. Esmaeilzadeh, Y. Kim, and K. Taghva, "A survey on mobile malware detection methods using machine learning," in *Proc. IEEE 12th Annu. Comput. Commun. Workshop Conf.*, 2022, pp. 0215–0221.

[33] E. Gelenbe, M. Nakıp, and T. Czachórski, "Improving massive access to IoT gateways," *Perform. Eval.*, vol. 157, 2022, Art. no. 102308.

[34] R. Subbaraman, Y. Guntupalli, S. Jain, R. Kumar, K. Chintalapudi, and D. Bharadia, "Bsma: Scalable lora networks using full duplex gateways," in *Proc. 28th Annu. Int. Conf. Mobile Comput. Netw.*, 2022, pp. 676–689.

[35] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," *ACM Trans. Intell. Syst. Technol.*, vol. 10, no. 2, pp. 1–19, 2019.

[36] G. Hinton et al., "Distilling the knowledge in a neural network," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2015, pp. 1–9.

[37] H. M. Choi, H. Kang, and D. Oh, "Unsupervised representation transfer for small networks: I believe i can distill on-the-fly," *Adv. Neural Inf. Process. Syst.*, vol. 34, pp. 24645–24658, 2021.

[38] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "Distilbert, a distilled version of BERT: Smaller, faster, cheaper and lighter," arXiv preprint 2019, *arXiv: 1910.01108*.

[39] N. Apthorpe, D. Reisman, and N. Feamster, "A smart home is no castle: Privacy vulnerabilities of encrypted IoT traffic," 2017, *arXiv: 1705.06805*.

[40] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997. [Online]. Available: https://doi.org/10.1162/neco.1997.9.8.1735

[41] J. M. Joyce, "Kullback-leibler divergence," in *International Encyclopedia of Statistical Science*, Berlin, Germany: Springer, 2011, pp. 720–722.

[42] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Representations*, 2015, pp. 1–15.

[43] A. Braeken, "Symmetric key based 5G aka authentication protocol satisfying anonymity and unlinkability," *Comput. Netw.*, vol. 181, 2020, Art. no. 107424.

[44] Y. Liu, L. Huo, and G. Zhou, "TR-AKA: A two-phased, registered authentication and key agreement protocol for 5G mobile networks," *IET Inf. Secur.*, vol. 16, no. 3, pp. 193–207, 2022.

[45] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. Mach. Learn. Syst.*, Austin, TX, USA, 2020, pp. 5132–5143.

[46] S. P. Karimireddy, S. Kale, M. Mohri, S. J. Reddi, S. U. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic controlled averaging for federated learning," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 5132–5143.

[47] D. A. E. Acar, Y. Zhao, R. M. Navarro, M. Mattina, P. N. Whatmough, and V. Saligrama, "Federated learning based on dynamic regularization," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 1–36. [Online]. Available: http://arxiv.org/abs/2111.04263

[48] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How to backdoor federated learning," in *Int. Conf. Artif. Intell. Statist.*, PMLR, 2020, pp. 2938–2948.

[49] Z. Zhu, J. Hong, and J. Zhou, "Data-free knowledge distillation for heterogeneous federated learning," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 12878–12889. [Online]. Available: http://arxiv.org/abs/2105.10056

Authorized licensed use limited to: King Mongkut's University of Technology Thonburi. Downloaded on October 13,2026 at 10:16:10 UTC from IEEE Xplore. Restrictions apply.



Yibing Liu (Member, IEEE) received the PhD degree from the State Key Laboratory of Mathematical Engineering and Advanced Computing, Information Engineering University, Zhengzhou, China, in 2023. He is currently a assistant research with the Defense Innovation Institute, Chinese Academy of Military Science, Beijing, China. His research interests include the next-generation mobile network, Internet of Things (IoT), machine learning, and information security.



Xiongtao Zhang received the BS and MS degree from the College of Systems Engineering at National University of Defense Technology, China, in 2018 and 2020, respectively. He is currently working toward the PhD degree with the College of Systems Engineering, National University of Defense Technology. His research interests include edge computing and FL.



Lijun Huo received the BS degree from the National University of Defense Technology (NUDT) of China, in 2018, and the master's degree in science and technology from Information Systems Engineering Laboratory, National University of Defense Technology, China. His research interests include neural networks, deep learning, reinforcement learning, 5G communication, and formal analysis.



Jun Wu (Senior Member, IEEE) recieved the PhD degree in information and telecommunication studies from Waseda University, Japan, in 2011. He is currently a professor with the Graduate School of Information, Production, and Systems with the Waseda University. He is the chair of IEEE P21451-1-5 Standard Working Group for Internet of things. His research interests include the intelligence and security techniques of Internet of Things (IoT), edge computing, Big Data, 5G/6G, etc. He is the author or co-author of more than 200 peer-reviewed journal/conference papers on the abovementioned topics. His publications have received a few distinctions, which includes the Best Paper Award of IEEE Transactions on Emerging Topics in Computing, in 2020, Best Paper Award of the International Conference on Telecommunications and Signal Process in 2019, Best Conference Paper Award of IEEE ComSoc Technical Committee on Communications Systems Integration and Modeling in 2018. He has served as the track chair for VTC 2019 and VTC 2020 and the TPC member of more than ten international conferences, including ICC, GLOBECOM, etc. He severs an associate editor for the *IEEE Systems Journal* and *IEEE Networking Letters*. He has served as a guest editor for *IEEE Transactions on Industrial Informatics*, *IEEE Transactions on Intelligent Transportation*, *IEEE Sensors Journal*, *Sensors*, *Frontiers of Information Technology & Electronic Engineering (FITEE)*, etc.



Mohsen Guizani (Fellow, IEEE) received the BS (with distinction), MS and PhD degrees in electrical and computer engineering from Syracuse University, Syracuse, NY, USA, in 1984, 1986, and 1990, respectively. He is currently a professor and associate provost with MBZUAI, Abu Dhabi, UAE. Previously, he worked in different institutions in the USA, University of Idaho, Western Michigan University, University of West Florida, University of Missouri, University of Colorado-Boulder, and Syracuse University. His research interests include wireless communications and mobile computing, applied machine learning, cloud computing, security and its application to healthcare systems. He was listed as a Clarivate Analytics Highly Cited researcher in computer science in 2019, 2020 and 2021. He has won several research awards including the "2015 IEEE Communications Society Best Survey Paper Award" as well 4 Best Paper Awards from ICC and Globecom Conferences. He is the author of nine books and more than 800 publications. He is also the recipient of the 2017 IEEE Communications Society Wireless Technical Committee (WTC) Recognition Award, the 2018 AdHoc Technical Committee Recognition Award, and the 2019 IEEE Communications and Information Security Technical Recognition (CISTC) Award. He served as the editor-in-chief of IEEE Network and is currently serving on the Editorial Boards of many IEEE Transactions and Magazines. He was the chair of the IEEE Communications Society Wireless Technical Committee and the chair of the TAOS Technical Committee. He served as IEEE Computer Society distinguished speaker and is currently IEEE ComSoc distinguished lecturer.

Authorized licensed use limited to: King Mongkut's University of Technology Thonburi. Downloaded on October 13,2026 at 10:16:10 UTC from IEEE Xplore. Restrictions apply.